

SocialMetrics AI

Rapport d'Évaluation du Modèle de Sentiment

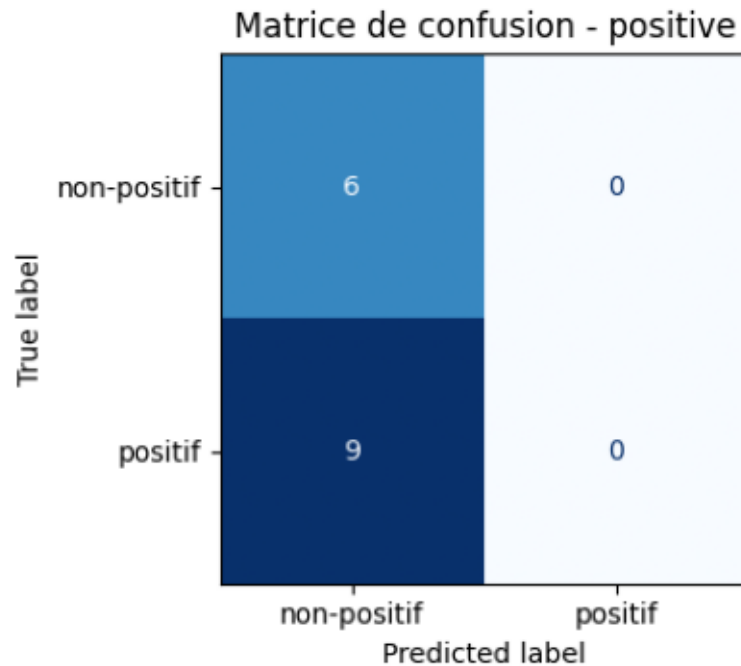
Client : Daunale Treupe

Ce rapport présente l'évaluation du modèle de régression logistique utilisé par l'API d'analyse de sentiments.

Le modèle est composé de deux classifieurs LogisticRegression indépendants (un pour le label "positive", un pour le label "negative") partageant un même vectoriseur TF-IDF (unigrammes + bigrammes, 5000 features max).

Jeu de données : 72 tweets annotés au total,
dont 57 utilisés pour l'entraînement et 15 pour la validation (split 80/20, random_state=42).

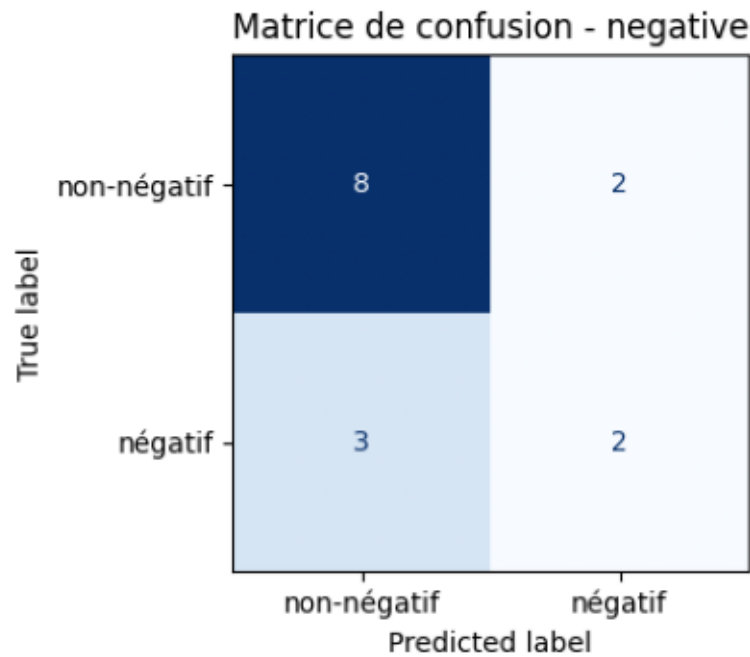
Matrice de confusion - Label positif



Interprétation :

Le modèle peine à identifier correctement les tweets positifs sur ce jeu de validation réduit : le rappel de la classe "positif" est très faible, la majorité des tweets positifs sont classés à tort comme "non-positif". Voir section Analyse pour les causes probables.

Matrice de confusion - Label negative



Interprétation :

Le modèle identifie correctement la majorité des tweets "non-négatif" et obtient un résultat correct sur la classe "négatif", bien que perfectible. Les performances restent limitées par la petite taille du jeu de données.

Précision, Rappel, F1-score

Label positive :

	precision	recall	f1-score	support
non-positif	0.40	1.00	0.57	6
positif	0.00	0.00	0.00	9
accuracy			0.40	15
macro avg	0.20	0.50	0.29	15
weighted avg	0.16	0.40	0.23	15

Label negative :

	precision	recall	f1-score	support
non-négatif	0.73	0.80	0.76	10
négatif	0.50	0.40	0.44	5
accuracy			0.67	15
macro avg	0.61	0.60	0.60	15
weighted avg	0.65	0.67	0.66	15

Analyse, Biais et Recommandations

Observations sur les performances

Le classifieur "negative" obtient des résultats corrects sur le jeu de validation, avec un F1-score raisonnable pour la classe négative. Le classifieur "positive" en revanche échoue à détecter la classe positive (rappel proche de 0) : il prédit quasi systématiquement la classe majoritaire "non-positif".

Biais identifiés

-
1. Taille du dataset : seulement ~72 tweets annotés, dont 15 en validation. Un jeu aussi réduit ne permet pas au modèle de généraliser, et la performance mesurée est très sensible au split train/validation.
 2. Déséquilibre de classes : sur un petit échantillon, la proportion positif/négatif/neutre dans le split de validation varie fortement, ce qui pénalise particulièrement la classe positive ici.
 3. Vectorisation TF-IDF avec bigrammes sur un vocabulaire restreint : risque de surapprentissage (mots vus uniquement dans un contexte précis à l'entraînement).
 4. Sensibilité aux accents/orthographe : le TF-IDF ne normalise pas les variantes sans accents ("genial" vs "génial"), ce qui peut créer des tokens hors-vocabulaire au moment de la prédiction.

Recommandations

-
1. Collecter un dataset annoté beaucoup plus large (plusieurs milliers de tweets) et représentatif du domaine réel (X / Twitter).
 2. Utiliser `class_weight="balanced"` dans `LogisticRegression` pour compenser le déséquilibre des classes.
 3. Valider avec une validation croisée (k-fold) plutôt qu'un split unique, pour obtenir des métriques plus stables.
 4. Normaliser le texte (minuscules, suppression des accents, lemmatisation) avant la vectorisation.
 5. Tester des modèles alternatifs ou complémentaires (embeddings pré-entraînés, modèles de langue) et comparer via validation croisée.
 6. Réévaluer régulièrement (à chaque réentraînement hebdomadaire) pour suivre la dérive des performances dans le temps.